Code-Layout, Readability and Reusability

Date	**02 October 2026**
Team ID	SWTID-2026-8237
Project Name	**Comic Craft - AI Comic Story Creator using Gemini Models**
Maximum Marks	**5 Marks**

Code Quality Checklist

S. No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the Python, HTML, CSS and Jinja2 code.	Yes	Consistent structure improves readability of routes, functions and templates.
2	Proper File Structure	Files and folders are logically organized into backend modules, templates and static output folders.	Yes	Modules such as routes.py, gemini_flash.py, gemini_pro.py, image_generator.py, layout_builder.py and exporters.py separate responsibilities.
3	Meaningful Variable Names	Variables reflect their purpose clearly.	Yes	Names such as story prompt, character name, setting, tone, art style and layout describe their roles.
4	Function / Method Names	Functions are descriptively named.	Yes	Generate _outline(), generate _story(), generate _image(), build _comic _layout() and save _pdf() clearly indicate their purpose.
5	Code Comments	Inline and block comments explain logic where required.	Partial	The project documentation explains the modules and workflow in detail; the supplied project material does not provide enough code-

				comment evidence to mark this fully Yes.
6	Modular Design	Code is split into reusable functions/modules.	Yes	Story generation, image generation, layout construction and PDF export are separated into dedicated modules.
7	No Redundant Code	Duplicate or unused code is removed.	Partial	The documented architecture is modular, but the supplied project document does not provide enough evidence for a complete code-level redundancy audit.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	Fast API route handlers use HTTP Exception and try-except blocks for API and JSON processing failures.

Reusable Components / Modules

S.No	**Component / Module Name**	**Language / Technology**	**Where Reused**	**Reusability Level (High / Medium / Low)**
1	Generate _outline()	Python / Gemini Flash	Called from the comic generation workflow to create the structured 5-panel outline.	High
2	Generate _story()	Python / Gemini Pro	Used after outline generation to create narration and character dialogue.	High
3	Generate _image()	Python / Stable Diffusion / Diffusers	Called for each comic panel and also exposed through /test-image for image-generation testing.	High

4	Build _comic _layout()	Python	Used to combine generated images and story content into panel layout data.	High
5	Save _pdf()	Python / FPDF	Used to compile the complete comic into a downloadable multi-page PDF.	High
6	routes.py	Python / Fast API	Coordinates form-based and JSON-based comic generation and connects reusable backend functions.	High
7	Jinja2 templates	HTML / Jinja2	index.html, comic_preview.html and export_success.html provide reusable dynamic rendering structure.	High

Code-Layout, Readability and Reusability:

This document evaluates the Comic Craft codebase structure based on the supplied project documentation. Comic Craft follows a modular design in which Fast API routes coordinate separate functions for outline generation, story generation, image generation, comic layout construction and PDF export. This separation makes the workflow easier to understand, maintain and reuse for future comic-generation features.

The project uses meaningful function names such as generate _outline(), generate _story(), generate _image(), build _comic _layout() and save _pdf(). The frontend is also separated into Jinja2 templates for input collection, comic preview and export confirmation. This organization supports readability and allows individual components to be reused by multiple routes or workflow stages.

Code Layout Checklist:

Backend routing is handled in routes.py.

AI text generation is separated into Gemini Flash and Gemini Pro modules.

Image generation is separated into image_generator.py.

Comic panel organization is handled by layout_builder.py.

PDF export is handled by exporters.py.

Frontend templates are separated into index.html, comic_preview.html and export_success.html.

Generated panel images and exported PDFs are stored in dedicated static folders.

Overall Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	The documented architecture separates routing, AI integration, image generation, layout and export responsibilities.
Readability	5	Descriptive function/module names and separate frontend templates make the workflow easy to follow.
Reusability	5	Core functions are independently callable and are reused across the comic-generation workflow and testing route.
Documentation / Comments	4	The supplied project documentation gives detailed module and workflow descriptions; direct source-code comment coverage is not fully shown.
Overall Score	5	Based on the supplied project documentation, the code organization demonstrates a strong modular structure suitable for maintenance and extension.

Overall Code Quality Assessment:

Comic Craft demonstrates a well-structured and modular application design. The separation of Fast API routing, Gemini-based text generation, Stable Diffusion image generation, panel layout construction and PDF export supports readability and reusability. The project documentation also shows clear separation of frontend templates and backend responsibilities. Based on the supplied project material, the strongest evidence is for modular design, meaningful function naming and reusable workflow components. Code-level comment coverage and redundancy cannot be fully verified from the supplied project document alone.